

ORK Graph: Architecture

What ORK Is

A learned cognitive substrate built around three structural commitments:

1. **Statements are decomposed before they enter the graph.** A single utterance becomes a set of atomic propositions. Each atom can be reasoned about, related, predicted-about independently.
2. **Edges are first-class learned vectors.** Not scalar weights, not predefined types. Each edge between two nodes is a vector in a shared edge-embedding space. The space is open-vocabulary: no fixed list of relation types.
3. **The training signal is relationship prediction.** Given two nodes (or a node and a context), predict the edge. Given a node and an edge, predict the other end. The graph improves by becoming better at this.

The point of a thinking organism is to make accurate predictions of the world around it, so that it can survive. ORK inherits this objective: minimise surprise on incoming statements, update structure when surprise is high. The graph earns its place in this story by doing what parametric models do poorly: holding long-tail facts sharp, factorising compositional structure, supporting sample-efficient learning by consulting external structure rather than baking everything into shared weights.

Why the Predecessor Failed

Descartes put all the semantics in the nodes. Propositions were embedded sentences sitting in an embedding space. Edges were traversal traces — a scalar count of how often the crawler had walked from A to B. Nothing about an edge said *what kind of relationship* connected its endpoints. The relationship was supposed to be implicit in embedding distance, bridge tokens, structural fingerprints.

Two structural problems with this:

There was no loss function. The four candidate signals (embedding distance, fingerprint similarity, bridge token novelty, density mismatch) were hand-tuned. The system ran, edges accumulated, the graph grew. But nothing pressured it toward any particular structure. “Productive” and “unproductive” traversals were defined by intrigue heuristics, not by whether the resulting graph predicted anything. The graph could not get better at its job because it had no job.

Edges as scalar weights could not carry relational meaning. The architecture quietly bet that everything important about a relationship could be recovered from the positions of the endpoint nodes plus a count. This works for “two ideas often co-

occur” but it fails the moment you need to distinguish *queens cause colonialism* from *queens oppose colonialism* — both connect the same two nodes, both are walked by the crawler, the resulting edges are indistinguishable.

ORK addresses both: edges are learned vectors, and the whole system is optimised against prediction.

Decomposition

A statement enters the system as text. Before it becomes graph structure it is passed through a decomposer that extracts atomic propositions. *The economy is struggling and people can't afford rent* decomposes to two atoms: *the economy is struggling*, *people can't afford rent*. Each becomes a node candidate or a (head, relation, tail) triple.

Decomposition is well-trodden in factuality evaluation — FActScore (Min et al., EMNLP 2023), SAFE (Wei et al., 2024, preprint), and the broader hallucination-detection literature have converged on LLM-prompted atomic-fact extraction as the standard tool. The technique can be taken as given.

What is not given is what counts as “atomic.” Atomicity is a design choice, not a natural kind. Davidsonian event semantics decomposes to predicate-argument structure with explicit event variables. RDF triples decompose to subject-predicate-object. AMR frames decompose to roleset-instantiated frames. Whole simple sentences are coarser. The right grain is the one at which the prediction objective produces the most useful structure — coarse enough that atoms are stable and re-encounterable, fine enough that distinct relationships do not get fused. Empirical, not philosophical.

For v0: triples extracted by an LLM (Claude / GPT-4o / open-weights equivalent), with the head and tail being entity mentions and the relation being free-text phrase.

Edges as Learned Vectors

Every edge in the graph is a vector in a shared edge-embedding space. Two edges that encode the same kind of relationship between different endpoints land near each other. Categories like *is-a-figurehead-of* or *prevents* exist as regions in this space without ever being declared. They emerge from data.

This is open-vocabulary knowledge graph completion. Closed-vocabulary versions are well-studied — TransE (Bordes et al., NIPS 2013), RESCAL (Nickel et al., ICML 2011), ComplEx (Trouillon et al., ICML 2016), RotatE (Sun et al., ICLR 2019), R-GCN (Schlichtkrull et al., ESWC 2018). The open-vocabulary variant is harder because the relation space has no fixed cardinality and no labels — the space must organise itself.

The bootstrap: initialise edge vectors using a pretrained relation encoder — a transformer that reads the (subject, relation phrase, object) triple and outputs a vector. The shared space comes pre-formed from the encoder’s pretraining. Edge

vectors then drift from these initialisations as the prediction objective applies pressure. This avoids the cold-start problem of trying to bootstrap a shared space from nothing.

Prediction as the Objective

The naive form of predictive processing — *predict the next token* — collapses to what language models already do. A graph buys nothing if it is just a worse LM. ORK has to earn its place by doing something parametric weights cannot:

- **Long-tail factual memory.** A fact mentioned once should remain sharply predictable. Parametric memory smears single-mention facts into noise. An explicit graph holds them.
- **Compositional structure.** If *queens are figureheads* and *Ronald McDonald is a figurehead* share an edge in the same region of edge-vector space, the system has factorised a generalisation. The LM has only entangled it across millions of weights.
- **Sample efficiency.** A predictor that consults a structured graph at inference time can learn from far fewer examples than one that has to bake everything into parameters.

The shape this implies is **retrieval-augmented prediction with a learned, structured memory**. Existing precedents in this neighbourhood: RETRO (Borgeaud et al., ICML 2022), GreaseLM (Zhang et al., ICLR 2022), QA-GNN (Yasunaga et al., NAACL 2021), kNN-LM (Khandelwal et al., ICLR 2020), Neural Theorem Provers (Rocktäschel & Riedel, NeurIPS 2017). None have decisively won. The bet here is that aggressive structuring — decomposed atoms with learned typed edges — produces a graph that supports more compositional generalisation than retrieving raw text spans does.

Relationships Over Tokens

The right thing to predict is not the next token. It is the relationship.

A baby's primal classifier is not generating sensory frames. It is asking *what edge connects me to this object?* Food, not-food. Hard, soft. Friend, foe. Fun, boring. The baby sticks a pencil in its mouth to find out *what kind of relationship* it has with the pencil. The category is the edge.

This reframes the training signal. The system encounters a new statement. The decomposer produces atoms. For each pair of atoms (or each atom and existing graph context), the system predicts the edge that should connect them. Loss is on this prediction. The model gets better at filling in *which relationship holds* given the structure it already has.

Categories emerge as clusters in edge-vector space. *Food* is a region. The baby's first edge to milk and its first edge to mother both initially land in the *pleasure* region. Later, when the system needs to predict different consequences from these two edges (one is consumed, one acts), the edges drift apart in the space. New regions appear as needed. No category is ever declared.

Affordances, not properties. “Is this food?” is really “can I eat this?” — Gibson’s affordances. The category is defined by what the object lets the agent *do*. This means relationships are minimally agent-relative. The graph likely needs a privileged *self* node, and many edges are implicitly statements about what the agent could do with the other end.

The action loop is load-bearing — and is the architecture’s biggest gap. A baby learns “soft” by pressing on things. The label is downstream of having performed the press and observed the deformation. Without an action loop, new categorical dimensions can only be grounded in textual co-occurrence — which is exactly what LMs already do well. Three honest options:

1. Treat reading and decomposing a statement as the “action” and the resulting graph update as the “feedback.” Weak but trivially available.
2. Give the system tool use. Let it query APIs, run code, fetch images. The action loop is real but the action space is narrow.
3. Accept that this is a disembodied symbolic baby. See what categories it can still discover from textual structure alone. Cheap experiment, tells you whether the architecture has legs before committing to embodiment.

1. is the most defensible long-term. (3) is the right first experiment.

Per-Edge Overfitting

Each edge can overfit to the examples that created it. This is not a bug. It is the architecture.

Standard ML worries about overfitting because every datapoint nudges every parameter, so noise from one example corrupts everything else. Per-edge overfitting does not have this problem. An edge that overfits to a single observation does not corrupt the rest of the graph. It just sits there, only consulted when the system asks about that exact pair of nodes. The failure mode *I made up a wrong generalisation* is replaced with *I do not know* — a much safer failure for a thinking system.

This is the **complementary learning systems** argument (McClelland, McNaughton & O’Reilly, Psychological Review 1995). The brain runs two memory systems for exactly this reason: the hippocampus deliberately overfits to single episodes; the neocortex slowly extracts statistical regularities through repeated exposure. Sleep consolidation is the bridge between them. The per-edge graph is the hippocampal end. A consolidation loop is what makes the cortical end possible.

Other precedents in the same neighbourhood:

- **Exemplar models** in cognitive psychology — Medin & Schaffer (Psychological Review, 1978), Nosofsky’s GCM (Journal of Experimental Psychology, 1986). Empirically defended single-instance storage against prototype/abstraction theories. Humans store episodes; generalisation happens at retrieval.
- **k-NN language models** (Khandelwal et al., ICLR 2020). LM plus a datastore of exact (context, next-token) pairs. Beats parametric-only models on long-tail prediction. Held back by cost, not by being wrong.
- **Neural Episodic Control** (Pritzel et al., ICML 2017). Explicit episodic memory in RL agents. Order-of-magnitude better sample efficiency.

- **Sparse Distributed Memory** (Kanerva, MIT Press 1988) and modern hyperdimensional computing.

Generalisation by retrieval, not by compression. This is the inversion. In an LM, generalisation is in the shared weights and individual facts are smeared. Here, individual facts are sharp and generalisation lives in the geometry of the edge space at query time. Asking *what is the relationship between baby and pen?* finds the cluster of soft-bitable-thing edges the baby built from pencils, crayons, fingers. The category exists as a region in edge-vector space without ever being named.

The shared edge space still has to come from somewhere. Either the pretrained relation encoder provides it (cheap, low commitment) or consolidation builds it — periodically train an edge encoder on the accumulated overfit edges so the shared space tracks what the system has actually learned.

Growing Dimensionality

A baby starts with one axis: pleasure-pain. Arguably nothing more. Every relationship the baby has with anything lives on that single axis. Milk is +pleasure. Hunger is -pleasure. Mother is +pleasure.

After a few months the axis becomes insufficient. The baby's relationship to milk and its relationship to mother are both maxed out on pleasure but they are not the same kind of thing. The system cannot predict the right responses with one axis. A new axis appears — call it love-pleasure, or agentic-presence, or whatever its discriminative role turns out to be. Mother and milk now sit in different places in the expanded space.

This is conceptual differentiation as Carey describes it in *The Origin of Concepts* (Oxford University Press, 2009). Children do not just accumulate facts. At certain points they undergo *qualitative reorganisations* where new representational primitives become available. The canonical example: pre-school children genuinely do not distinguish weight from density. Around age 7–8 these collapse to one axis until experience forces a split. Carey calls this “Quinian bootstrapping.”

The formalisation already exists. **Indian Buffet Process** (Griffiths & Ghahramani, NIPS 2005) — a nonparametric Bayesian prior over feature matrices with an unbounded number of latent dimensions. Each observation either uses existing features or “orders new dishes from the buffet.” The model decides how many axes it needs from the data.

What is novel here against the existing IBP / nonparametric literature is that **developmental order matters**. In standard nonparametric models, you collect all your data and infer the right number of latents post-hoc. Here, the order axes come online matters because each new axis is built on top of the structure that existed when it appeared. A system that develops love-pleasure as axis two has a different cognitive topology than one where the axes are inferred jointly from a complete dataset.

The trigger for adding an axis is discriminative collision. Two edges that produce identical vectors in the current space but should behave differently — the baby's relationship to milk and to mother both collapsing to +pleasure but predicting

different outcomes. The system makes wrong predictions. Add an axis whose job is to separate them. Every new dimension has a discriminative role and (in principle) a name.

This is preferable to two alternatives the literature uses more often:

- **Reconstruction failure** as trigger — when residual prediction error on existing dimensions plateaus, add a new dimension to absorb the residuals. Principled but does not tell you what the new axis is *for*.
- **Minimum description length** as trigger — add an axis when the cost of representing entanglement in the current space exceeds the cost of a new dimension. Cleanly Bayesian but hard to operationalise.

Discriminative collision is more interpretable and matches the developmental story: dimensions appear when the agent needs them to predict, not abstractly when variance accumulates.

Old edges need a projection rule. When the love-pleasure axis comes online, every existing edge implicitly had a 0 on that axis. Some genuinely should — the relationship to a wooden block has zero love content. Some should be retroactively non-zero — the relationship to mother was always partly on the love axis, the axis just did not exist yet. A backfit policy is required. The brain handles this through replay; ORK handles it through the consolidation loop.

Orthogonality is not automatic. New axes added greedily will be correlated with existing ones. “Love” and “pleasure” really do overlap a lot. Two options: accept correlation (cortical dimensions are entangled empirically; correlated axes still add discriminative power) or impose orthogonality pressure (cleaner inspection, but forces the model to keep re-explaining variance other axes could have handled). Modest orthogonality pressure is probably the right tradeoff for observability.

The Minimum Publishable Subset

Of the four research threads in this architecture (edge prediction with decomposition; per-edge overfitting and consolidation; dimensional growth; affordance-grounded action loops), only one is thesis-shaped enough to ship first:

Open-vocabulary learned edge prediction with claim decomposition. Clean problem statement, established benchmarks, comparable baselines, publishable in increments. The other three live in future-work chapters and in conversation.

Pipeline

1. **Text corpus in** (Wikipedia paragraphs, books — anything with factual content).
2. **Decomposer:** each paragraph → list of atomic claims. LLM-prompted with structured output. Decomposer is a tool, not the contribution.
3. **Triple extraction:** each atom → (head, relation_phrase, tail). Relation phrase is raw text, not a label from a fixed vocabulary.
4. **Encoders:**
 - *Entity encoder:* pretrained sentence transformer over the entity mention → vector. Frozen + small projection head.
 - *Edge encoder:* same or separate pretrained transformer over the relation phrase → vector. Also frozen + projection.

5. **Graph store:** (head_vector, edge_vector, tail_vector) triples. Edge vectors live in a shared space — this is where open-vocabulary generalisation lives.
6. **Scoring function:** DistMult-style ($\langle h, r, t \rangle$ elementwise dot product) for v1. Bilinear is more expressive but blows up parameter count. ComplEx / RotatE with complex embeddings is a strong baseline for closed-vocab — adaptable to open-vocab but trickier.
7. **Training:** margin ranking loss with negative sampling. Hard negatives matter. In-batch negatives as the cheap default; add a hard-negative miner once the basic loop works.

Open-Vocabulary Mechanism

In closed-vocab KGE, every relation type has its own learned embedding from a fixed table — `embedding_table[relation_id]`. Adding a new relation type at inference is impossible without retraining.

Here, the edge vector comes from `edge_encoder(relation_phrase)`. The encoder is a pretrained text model that can encode any phrase. So are figureheads of and serve as figureheads of produce nearby vectors — the encoder already knows they mean roughly the same thing. The training loss refines where these vectors sit relative to each other in a way that makes the scoring function work.

Baselines

- **Closed-vocab KGE:** TransE, ComplEx, RotatE, R-GCN. Standard FB15k-237 / WN18RR comparison. Table stakes.
- **Universal Schema** (Riedel et al., NAACL 2013). Open-vocab predecessor.
- **Similarly-sized pretrained LM, zero-shot and few-shot.** GPT-2 small (124M), Pythia-160M / Pythia-410M — methodologically clean because Pythia’s training data is documented. Prompt with (head, relation phrase, ?) and score the true tail’s rank.
- **Similarly-sized LM fine-tuned on the same training triples.** LoRA or continued pretraining. Tests the architecture against a directly-supervised LM with matched parameters.
- **k-NN over training triples — the critical baseline.** Embed every training triple with the encoder. At test time, retrieve nearest-neighbour training triples and predict from them. *No learned graph structure at all.* If ORK doesn’t beat k-NN, the learned edge structure isn’t earning its keep.
- **Sanity:** random ranker, majority-class.

Frontier LMs (GPT-4, Claude) are deliberately excluded — their parameter counts aren’t disclosed, making “fair comparison” impossible, and the comparison degenerates into “more compute beats less compute” rather than testing architecture.

Datasets

- FB15k-237 — closed-vocab competitiveness.
- WN18RR — closed-vocab competitiveness, lexical relations.
- T-REx — open-vocab Wikipedia-derived.
- Custom: a Wikipedia subset decomposed by the v0 pipeline, with held-out triples for evaluation.

Metrics

MRR, Hits@1, Hits@10 — KG completion standards. Plus a zero-shot relation evaluation (test relations never appear in training) that closed-vocab baselines cannot run.

V0 Loop

Smallest possible end-to-end thing:

1. 10k Wikipedia paragraphs, decomposed by an LLM.
2. Frozen sentence-transformers/all-MiniLM-L6-v2 for both entity and relation encoding, with a single projection layer each.
3. DistMult scoring, margin ranking loss with in-batch negatives.
4. Train, evaluate on a 1k held-out paragraphs.
5. Read failure cases. Iterate.

The point of v0 is not to beat baselines — it's to confirm the loop works and look at what the model is actually doing. Failure analysis on a small dataset tells you what to fix before scaling up.

Open Problems

The action loop. Affordance-grounded learning needs interaction. Pure text co-occurrence is what LMs already do; if the graph adds nothing beyond that channel it is not earning its keep. Tool use is the most realistic answer but the action space is narrow compared to an embodied agent.

Backfit policy during consolidation. When a new edge dimension appears, which old edges get re-projected and how. Naive options: re-encode all of them (expensive but principled), re-encode only the ones the consolidation pass touches (cheaper, biased toward recent material), leave them and rely on similarity at query time to fill in (cheapest, lossy). All three are defensible.

Cold start of the edge encoder. Pretrained relation encoders are the obvious bootstrap. The risk is that the pretrained space is structured around human-named relations, biasing the system toward those categories before it has a chance to discover its own. Whether this matters in practice depends on how strongly the prediction objective can deform the pretrained space.

Observability without sacrificing the loss. A graph trained by prediction error converges on whatever shortcuts minimise loss, not necessarily on the human-readable structure we want to inspect. Interpretability research keeps hitting this. Picking prediction as the objective means accepting that the graph might be alien — encoding whatever co-occurrence patterns reduce surprise, not the typed relations a human would have hand-built. That is fine, maybe the point, but worth knowing going in.

The consolidation loop. Carries more weight here than it did in Descartes. It does specific structural work: training the edge encoder on accumulated overfit edges, backfitting old edges into newly grown dimensions, extracting from hippocampal-style episodic storage to neocortical-style compressed structure. If consolidation does not work, the architecture does not work.

Prior Art

Knowledge graph embeddings (closed-vocabulary): TransE (Bordes et al., NIPS 2013), RESCAL (Nickel et al., ICML 2011), ComplEx (Trouillon et al., ICML 2016), RotatE (Sun et al., ICLR 2019), R-GCN (Schlichtkrull et al., ESWC 2018).

Open-vocabulary relation learning: Universal Schema (Riedel et al., NAACL 2013), OpenIE-series (Banko et al., IJCAI 2007 onwards).

Atomic claim decomposition: FActScore (Min et al., EMNLP 2023), SAFE (Wei et al., 2024, preprint).

Retrieval-augmented prediction: RETRO (Borgeaud et al., ICML 2022), kNN-LM (Khandelwal et al., ICLR 2020), Neural Theorem Provers (Rocktäschel & Riedel, NeurIPS 2017), GreaseLM (Zhang et al., ICLR 2022), QA-GNN (Yasunaga et al., NAACL 2021).

Complementary learning systems and exemplar memory: McClelland, McNaughton & O'Reilly (Psychological Review, 1995); Medin & Schaffer (Psychological Review, 1978); Nosofsky (Journal of Experimental Psychology, 1986); Pritzel et al. (Neural Episodic Control, ICML 2017); Kanerva (Sparse Distributed Memory, MIT Press 1988).

Nonparametric latent factor learning: Indian Buffet Process — Griffiths & Ghahramani (NIPS 2005); Chinese Restaurant Process and Dirichlet Process priors for unbounded mixtures (Ferguson, 1973; Antoniak, 1974).

Developmental conceptual change: Carey, *The Origin of Concepts* (Oxford University Press, 2009); Kelly's Personal Construct Theory (1955); Spelke on core knowledge; Lake & Tenenbaum on concept learning.

Predictive processing: Friston (free energy principle, *Nature Reviews Neuroscience* 2010 onwards); Clark, *Surfing Uncertainty* (Oxford University Press, 2016); Hawkins, *A Thousand Brains* (Basic Books, 2021); LeCun's world model / JEPA position paper (2022, preprint).

Affordances: Gibson, *The Ecological Approach to Visual Perception* (Houghton Mifflin, 1979).